

Configuration Proof

1. High Level Conclusion

Branch 1 (the slow path, for real proposals)

Proposer Safe (5/11) --module--> Delay (+ PauseGuard) --module--> Ownerless Safe

Branch 2 (the token-vote path, whose only power is to veto)

Governor --module--> Roles (+ SetTxNonceGuard) --target--> DelayOwnerSafe --owner--> Delay

Pause controls

PauseSafe (2/9) --pause--> PauseGuard

Admin Safe --unpause / setPauser--> PauseGuard

The summation of all of the conclusions drawn from FV proofs should prove the following generalizations:

1. Only the Term multisig holders on the proposer safe and ownerless safe may execute DEVOPS_ROLE methods holding: 1. the owners, fallbackHandler, modules, guard are constant for the Proposer Safe and Ownerless Safe, and 2. The owner, target, modules, guard, txCooldown, and txExpiration are held constant for the Delay Modifier.
2. Only TERM token holders and Term multisig holders may veto Delay modifier transactions, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard are held constant on the Delay and Roles Modifiers, and 3. the role targets, role scopes, and role assignments are also held constant on the Roles Modifier.
3. The following: 1. the owners, fallbackHandler, modules, guard for the ProposerSafe, OwnerlessSafe, and DelayOwnerSafe, and 2. the owner, target, modules, guard for the Delay Modifier, and 3. the owner, target, modules, guard, role targets, role scopes, and role assignments for the Roles Modifier, cannot be changed other than by the Term multisig holders on the ownerless safe, proposer safe, or DelayModOwner safe.

Only Term multisig holders can execute DEVOPS_ROLE methods subject to TERM token governance, and this can't be changed other than through Term multisig approval.

2. Evidence Classification

This document uses two classes of evidence to establish the security properties of the proposed governance configuration:

- **Formal evidence** establishes that, assuming the stated configuration, the relevant contracts enforce the claimed security properties. This includes proofs about privileged entry points, authorized callers, configuration immutability, and the absence of alternative execution paths. In the tables below, formal evidence is labeled **FV** — and names each rule;  marks a rule that passes.
- **Deployment evidence** establishes that the deployed contracts actually have the configuration assumed by the formal evidence. This includes on-chain verification of contract addresses, owners, modules, guards, thresholds, roles, and other relevant state, each read at a stated block. In the tables below, deployment evidence is labeled **On-chain** —;  marks a read that matches the stated configuration.

These two classes of evidence are complementary. Formal evidence does not by itself establish that the deployed contracts have the configuration being proved, and deployment evidence does not by itself establish that the deployed configuration enforces the claimed security properties.

Configuration items marked  (pending migration) describe the intended post-migration configuration and are not established by the current deployment evidence. These items must be re-verified against on-chain state after the migration is executed.

3. Evidence

Generalization 1 — Only the Term multisig holders on the proposer safe and ownerless safe may execute DEVOPS_ROLE methods holding: 1. the owners, fallbackHandler, modules, guard are constant for the Proposer Safe and Ownerless Safe, and 2. The owner, target, modules, guard, txCooldown, and txExpiration are held constant for the Delay Modifier.

#	Statement	Evidence
G1.1	Only the Ownerless Safe holds the DEVOPS_ROLE required for protocol changes.	On-chain — the Ownerless Safe holds DEVOPS_ROLE on each core contract: ✅ at block 26080131, DEVOPS_ROLE (keccak256("DEVOPS_ROLE") = 0x793a6c9b7e0a9549c74edc2f9ae0dc50903dfaa9a56fb0116b27a8c71de3e2c6) is held by the Ownerless Safe (0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03) on: • TermController (0xd902EBb8AEb832643af38f43d466a6155cD8Bd5A), read by hasRole • TermEventEmitter (0xf021B31282a60528B2F47D07Ce353da870be78b3), read by hasRole • TermPriceConsumerV3 (0x13Ca4ddB295d621761057D682a1c7b5f5D7Bba4c), read by hasRole • TermRepoDeployerFactory (0x44058C32B154A516F3b9DE2413e6cb938F93c3B4), read by hasRole • TermInitializer (0x3B668F408293e9a5700D6Eeae78E1f945CF501eF), read by hasRole • TermDiamond (0xb4B1de03F229220eDbBB31A224b2a004b3c3e47f), read by storage slot 0x5fdec70149aea3337a70dda36f71a4911e25e15ef0e98916a983b6ff4c6aa5a4 = 1 On-chain — it has only ever been granted to the Ownerless Safe: ✅ Blockscout's full log history for each contract holds exactly one event naming DEVOPS_ROLE: a RoleGranted to the Ownerless Safe, emitted in the contract's own creation transaction. There is no RoleRevoked, no RoleAdminChanged, and no grant to any other address. • TermController: block 25188635, tx 0xdeea44228ef14d20a66a02e0a3d84907bcd854aa0f352a5ad8233b48b23f7f15 • TermEventEmitter: block 25188643, tx 0x15c3af7845f086fca8a069ea755f92f4613ae716978aa100089788d98a68d29e • TermPriceConsumerV3: block 25188652, tx 0x7b960b5c31408ac8cf6eb77766c2c8f747bf74b9f2ec1fd98f1d66999a3d6063 • TermRepoDeployerFactory: block 25188657, tx 0x22c72baf2cf6a6ad808e874ca9d75190592059fa9e0591451ea738147394f8f5 • TermInitializer: block 25555701, tx 0xec1c9e2935569e769123e7a05f6d1f5fc2c1ba7aa2a7b4013d799dfee76885d7 • TermDiamond: block 25188499, tx 0xe70a739ab7ff78a6b8ae8beefd002c7ee4a87d8c41b6ec594ae45eba87412788 On-chain — nobody can grant additional DEVOPS_ROLE holders with grantRole: ✅ on the five AccessControl contracts, getRoleAdmin(DEVOPS_ROLE) is DEFAULT_ADMIN_ROLE (0x00). No

#	Statement	Evidence
		production contract grants <code>DEFAULT_ADMIN_ROLE</code> or calls <code>_setRoleAdmin</code>, and no <code>RoleGranted</code> for <code>DEFAULT_ADMIN_ROLE</code> appears in any of the six histories, so no address can call <code>grantRole(DEVOPS_ROLE, ...)</code>. On the <code>TermDiamond</code>, <code>grantRole</code> is not registered at all: its roles are set once by <code>initDiamond</code>, or by a new facet added through <code>diamondCut</code>, which is itself <code>DEVOPS_ROLE</code>-only.
G1.2	Only Modules or Owners can interact with <code>OwnerlessSafe</code> , except <code>fallback</code> and <code>receive</code> , which anyone can call.	FV — every entry point to the Safe is enumerated below: SE-1 <code>safeEntryPointsAreAllAccountedFor</code>  passes. Safe v1.3.0 has no write functions besides these seventeen entry points (the Prover checks <code>fallback</code> and <code>receive</code> as one entry), and any other call lands in <code>fallback</code>. Grouped by who can call each one: <i>module gated entry points:</i> <code>execTransactionFromModule</code> <code>execTransactionFromModuleReturnData</code> <i>owner gated entry points:</i> <code>execTransaction</code> <code>approveHash</code> <i>Self calls — only modules and owners can reach these:</i> <code>enableModule</code> <code>disableModule</code> <code>addOwnerWithThreshold</code> <code>removeOwner</code> <code>swapOwner</code> <code>changeThreshold</code> <code>setGuard</code> <code>setFallbackHandler</code> <i>Non-interactions — these always revert, so nothing they do stands:</i> <code>setup</code> <code>requiredTxGas</code> <code>simulateAndRevert</code> <i>Open to anyone:</i> <code>fallback</code> <code>receive</code> FV — module-only entry points can make the Safe execute to any destination: <code>execTransactionFromModule</code>, <code>execTransactionFromModuleReturnData</code>. - SE-3 <code>onlyEnabledModulesCanCallModuleExec</code>  passes: only an enabled module can successfully call <code>execTransactionFromModule</code> or <code>execTransactionFromModuleReturnData</code>. - SE-4 <code>enabledModuleCanMakeTheSafeAct</code>  passes: an enabled module can call <code>execTransactionFromModule</code> and <code>execTransactionFromModuleReturnData</code> directly, with no owner signatures, and the Safe makes an outgoing call or delegatecall. FV — owner-only entry points can make the Safe execute to any destination: <code>execTransaction</code>, <code>approveHash</code>. - SE-5 <code>execTransactionRunsOnlyAfterTheSignatureCheck</code>  passes: <code>execTransaction</code> only runs once the signature threshold check has passed for that exact transaction. - SE-12 <code>eachSignatureAcceptsANewApprovingOwner</code>  passes: every signature the check accepts is a different owner who approved it. With SE-5,

#	Statement	Evidence
		the Safe acts through <code>execTransaction</code> only when as many different owners as the threshold have approved. • SE-8 <code>onlyOwnersCanApproveHashes</code> ✓ passes: <code>approveHash</code> only records an owner's own approval, which then counts as that owner's signature in <code>execTransaction</code>. • SE-6 <code>ownersCanMakeTheSafeAct</code> ✓ passes: once the threshold is passed, <code>execTransaction</code> succeeds and the Safe acts, for any threshold and any number of owners. FV — settings functions are only reached through the module-only or owner-only points: • SE-7 <code>settingsOnlyChangeWhenTheSafeCallsItself</code> ✓ passes: these only change a setting when the caller is the Safe itself. • SC-4 <code>settingsOnlyChangeThroughAModuleOrOwners</code> ✓ passes: over every write function, <code>fallback</code> included, a setting only changes through an enabled module's <code>execTransactionFromModule</code> or <code>execTransactionFromModuleReturnData</code>, or through <code>execTransaction</code>. A fallback handler is checked as if it were the Safe itself, so it cannot reach these functions even then. • SC-1 <code>moduleCanChangeSettings</code>, SC-2 <code>moduleCanChangeSettingsWithReturnData</code> ✓ pass: for each function above, a module's <code>execTransactionFromModule</code>, and likewise its <code>execTransactionFromModuleReturnData</code>, addressed to the Safe with a call to that function, succeeds and changes a setting. • SC-3 <code>ownersCanChangeSettings</code> ✓ passes: the same through <code>execTransaction</code>, once the threshold is passed. FV — setup is not a callable function after deploy: SE-9 <code>setupAlwaysRevertsAfterSetup</code> ✓ passes. Once the Safe is set up, <code>setup</code> always reverts, so nobody can re-run it to replace the owners, threshold, modules or handler. FV — <code>requiredTxGas</code> and <code>simulateAndRevert</code> always revert: SE-10 <code>requiredTxGasAlwaysReverts</code>, SE-11 <code>simulateAndRevertAlwaysReverts</code> ✓ pass. Anyone can call them, and they do execute a call (<code>requiredTxGas</code>) or <code>delegatecall</code> (<code>simulateAndRevert</code>), but they always revert afterwards, so nothing they do stands. FV — Any address can call any unsupported Safe function to initiate <code>fallback</code>, which can only make the Safe execute to Fallback Handler address if set to nonzero address: • SE-14 <code>fallbackOnlyCallsItsHandler</code> ✓ passes: with a handler set, <code>fallback</code> only makes a plain call, with no ETH, to the handler. It never <code>delegatecalls</code>, so the handler's code runs as its own contract, not as the Safe, and cannot write the Safe's storage. Assumes the handler is not the Safe itself: v1.3.0 allows that (v1.4.0 forbids it, GS400), and then <code>fallback</code> calls back into the Safe as the Safe. That case is covered by SE-16. • SE-15 <code>fallbackHandlerCallsComeFromTheHandler</code> ✓ passes: once <code>fallback</code> calls its handler, every call has <code>msg.sender=handler</code>. Assumes the handler is not the Safe itself; that case is SE-16. • SE-16 <code>selfHandlerFallbackChangesNothing</code> ✓ passes: with the Safe set as its own handler, <code>fallback</code> leaves the Safe's storage exactly as it was, and when it succeeds the Safe called nothing but itself, sent no ETH and made no <code>delegatecall</code>. • SE-13 <code>fallbackMakesNoCallWithoutAHandler</code> ✓ passes: with no handler set (the zero address), <code>fallback</code> makes no call or <code>delegatecall</code> at all.

#	Statement	Evidence
		FV — Any address can trigger <code>receive</code> by sending Eth without calldata and it does nothing but emits <code>SafeReceived</code>: • SE-17 <code>receiveOnlyEmitsSafeReceived</code>  passes: whenever <code>receive</code> runs, it does nothing beyond the event. It emits exactly one log, <code>SafeReceived</code>, naming the caller as sender. It makes no call of any kind (call, delegatecall, staticcall or callcode) and writes no storage. • SE-18 <code>receiveCanRun</code>  passes: some call to the Safe succeeds and emits <code>SafeReceived</code>, so SE-17 is not vacuous.
G1.3	OwnerlessSafe fallback handler is <code>address(0)</code> .	On-chain — handler is <code>address(0)</code>:  incomplete until the governance reconfiguration plan is applied. The fallback handler slot (<code>0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5</code>) on the Ownerless Safe (<code>0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03</code>) must be zero. At block 26085317 it holds <code>0xf48f2B2d2a534e402487b3ee7C18c33Aec0Fe5e4</code> (<code>CompatibilityFallbackHandler v1.3.0</code>).
G1.4	OwnerlessSafe owners are Term multisig holders.	On-chain — nine keys, all nine needed:  at block 26084853, the Ownerless Safe (<code>0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03</code>) runs GnosisSafe v1.3.0 (<code>VERSION</code>), has threshold 9 (<code>getThreshold</code>) and these 9 owners (<code>getOwners</code>), each an externally owned account with no contract code: • <code>0xc1047a4D9f6071B55585523F7F659F39A1fBA74b</code> • <code>0x8EF485fD38b7B29a827938C9F00300eaBf8E1710</code> • <code>0xbfFcAdCd5549cC378693108BcD4435776A6fa795</code> • <code>0xB680373c50E9E877DA7dCF9efcc0dFB234452ad3</code> • <code>0x6eb0c274EC4B1d51152e45BAED22f883B2A3Bc60</code> • <code>0xE82183cfAE1C24f044315c318FD97bE7e47b31D2</code> • <code>0xeeee661edcFE634Dc0e29D62C26AfD62c0843b817</code> • <code>0xDB97c377F23a9Dc54c3D6F9df7bC0ca4Cf7D5A81</code> • <code>0x89562FC5AEF155481aE0C7dde1300110B4dF2F1D</code> FV — the owner set only changes through the Safe itself: SE-7 <code>settingsOnlyChangeWhenTheSafeCallsItself</code>, SC-4 <code>settingsOnlyChangeThroughAModuleOrOwners</code>  pass. <code>addOwnerWithThreshold</code>, <code>removeOwner</code>, <code>swapOwner</code> and <code>changeThreshold</code> only take effect when the Safe calls itself, through <code>execTransaction</code> (all nine owners, SE-5, SE-12) or its only module, the Delay (G1.5). SE-9 <code>setupAlwaysRevertsAfterSetup</code>  passes: setup cannot be re-run to replace them. Who holds the nine keys is not visible on chain. That they are Term multisig holders is a deployment fact.
G1.5	Delay Module is the only enabled module on OwnerlessSafe.	On-chain — the Delay is the only module:  incomplete until the governance reconfiguration plan is applied. <code>getModulesPaginated</code> on the Ownerless Safe (<code>0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03</code>) must return only the Delay (<code>0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf</code>). At block 26085317 it returns two modules: the old Roles (<code>0x405b47354CF06A25DE1DDb35EC65F03939E2e8D2</code>) and the Delay. Section 5 of <code>new_governance_plan_ownerless_safe.md</code> removes the old Roles, after which this can be re-read.
G1.6	Only Modules or Owner can interact with Delay Modifier, except <code>executeNextTx</code> and <code>skipExpired</code> , which anyone can call.	FV — every entry point to the Delay Modifier is enumerated below: DM-13 <code>delayWriteFunctionsAreTheKnownFifteen</code>  passes. The Delay has no fallback and exactly fifteen write functions, each covered below: • <code>setTxCooldown</code> — owner only (DM-8) • <code>setTxExpiration</code> — owner only (DM-8)

#	Statement	Evidence
		• <code>setTxNonce</code> — owner only (DM-8) • <code>setAvatar</code> — owner only (DM-8) • <code>setTarget</code> — owner only (DM-8) • <code>enableModule</code> — owner only (DM-8) • <code>disableModule</code> — owner only (DM-8) • <code>setGuard</code> — owner only (DM-8) • <code>transferOwnership</code> — owner only (DM-8) • <code>renounceOwnership</code> — owner only (DM-8) • <code>execTransactionFromModule</code> — modules only (DM-4) • <code>execTransactionFromModuleReturnData</code> — modules only (DM-4) • <code>executeNextTx</code> — anyone (DM-11) • <code>skipExpired</code> — anyone (DM-12) • <code>setUp</code> — nobody after deployment (DM-1) FV — owner-only settings: DM-8 <code>atMostOneCallerPassesOnlyOwner</code>  passes. Only the owner can call the Delay's ten settings functions. There can only be one owner at a time. FV — module-only queueing: DM-4 <code>queueOnlyGrowsThroughEnabledModules</code>  passes. Only an enabled module can add a transaction to the queue. FV — setup is closed: DM-1 <code>setUpAlwaysRevertsAfterDeployment</code>  passes. Nobody can re-run the Delay's setup after deployment. FV — the two exceptions are open: DM-11 <code>anyoneCanExecuteNextTx</code>, DM-12 <code>anyoneCanSkipExpired</code>  pass. An address that is neither the owner nor a module can execute a queued transaction and skip an expired one. FV — every other function, checked all at once: DM-14 <code>onlyModulesOrOwnerCanCallDelay</code>  passes. For every write function except <code>executeNextTx</code>, <code>skipExpired</code> and <code>setUp</code>, a call that succeeds came from the owner or an enabled module. It does not rely on a list of names, so it also covers any function added later.
G1.7	Only enabled modules can queue transactions to Delay Modifier. Therefore, Delay Modifier's owner, <code>DelayOwnerSafe</code> , cannot queue transactions to Delay Modifier.	FV — only an enabled module can queue: DM-4 <code>queueOnlyGrowsThroughEnabledModules</code>  passes. Over every write function of the Delay except <code>setUp</code> (closed by DM-1), the queue nonce and every queue entry only change when the caller is an enabled module. Being the owner gives no way in: none of the owner's ten settings functions (DM-8) adds to the queue. On-chain — the owner is not an enabled module:  at block 26084775, <code>isModuleEnabled</code> on the Delay (<code>0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf</code>) returns false for its current owner, the old Roles (<code>0x405b47354CF06A25DE1DDb35EC65F03939E2e8D2</code>), and for its planned owner, the <code>DelayOwnerSafe</code> (<code>0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3</code>). Its only module is the <code>Proposer Safe</code> (G1.8).
G1.8	<code>ProposerSafe</code> is the only enabled module on the Delay Mod	On-chain — the Delay's module list contains only the Proposer Safe:  asked for all of its modules, the Delay (<code>0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf</code>) returns just one, the <code>Proposer Safe</code> (<code>0xd5E12854A3DBa99deF295A7635D3Ba16427d2A28</code>), and reports that the list ends there (<code>getModulesPaginated</code>, block 26080101).

#	Statement	Evidence
		On-chain — the Delay's storage holds only the Proposer Safe as a module:  read directly from storage rather than through the Delay's own functions, the module list starts at the Proposer Safe and the Proposer Safe points back to the start, so it is the only entry (modules, storage slot 104, block 26080101).
G1.9	Only Modules or Owners can interact with ProposerSafe, except fallback and receive, which anyone can call.	Same as G1.2: the Proposer Safe runs the same GnosisSafe v1.3.0 code as the Ownerless Safe.
G1.10	ProposerSafe fallback handler is address(0).	On-chain:  the fallback handler slot (0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5, FallbackManager.sol:12) on the Proposer Safe (0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28) is zero at block 26056133.
G1.11	ProposerSafe owners are Term multisig holders.	On-chain — eleven keys, any five act:  at block 26084863, the Proposer Safe (0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28) runs GnosisSafe v1.3.0 (VERSION), has threshold 5 (getThreshold) and these 11 owners (getOwners), each an externally owned account with no contract code: • 0xF6A745f9B38FFcd17Ee8909AC89151D788F95282 • 0xC43d527E3544A3d199Ce28E87994384D89d90e6E • 0x8EF485fD38b7B29a827938C9F00300eaBf8E1710 • 0xbFfcAdCd5549cC378693108BcD4435776A6fa795 • 0xB680373c50E9E877DA7dCF9efcc0dFB234452ad3 • 0x6eb0c274EC4B1d51152e45BAED22f883B2A3Bc60 • 0xE82183cfAE1C24f044315c318FD97bE7e47b31D2 • 0xeeee661edcFE634Dc0e29D62C26AfD62c0843b817 • 0xc1047a4D9f6071B55585523F7F659F39A1fBA74b • 0xDB97c377F23a9Dc54c3D6F9df7bC0ca4Cf7D5A81 • 0x89562FC5AEF155481aE0C7dde1300110B4dF2F1D FV — the owner set only changes through the Safe itself: SE-7 settingsOnlyChangeWhenTheSafeCallsItself, SC-4 settingsOnlyChangeThroughAModuleOrOwners  pass. addOwnerWithThreshold, removeOwner, swapOwner and changeThreshold only take effect when the Safe calls itself, and with no modules (G1.12) that means through execTransaction, with 5 of the 11 owners (SE-5, SE-12). SE-9 setupAlwaysRevertsAfterSetup  passes: setup cannot be re-run to replace them. Who holds the eleven keys is not visible on chain. That they are Term multisig holders is a deployment fact.
G1.12	ProposerSafe has no enabled modules.	On-chain:  getModulesPaginated on the Proposer Safe (0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28) returns an empty list at block 26050570.
G1.13	ProposerSafe can propose a transaction to Delay Modifier that is executed on the Ownerless Safe once the cooldown has passed, if it hasn't expired, the system isn't paused, and it hasn't been vetoed.	FV — the Proposer Safe can propose to the Delay Modifier as a module: DM-5 enabledModuleCanQueue  passes. Any enabled module can queue a transaction on the Delay Modifier: its call to execTransactionFromModule, with any destination, value, calldata and operation, succeeds and adds the transaction to the queue. The Proposer Safe is the Delay Modifier's only enabled module (G1.8). FV — transaction proposed by module executes after cooldown and before expiration: DP-29 queuedTransactionExecutesAfterCooldown  passes. Once a module has queued a transaction, and the cooldown has

#	Statement	Evidence
		passed, it hasn't expired and the system isn't paused, anyone can execute it and it reaches the Ownerless Safe. Holds for every transaction, assuming the Ownerless Safe accepts the call. FV — transaction cannot execute too early or too late: DP-30 <code>executeNextTxRevertsDuringCooldown</code>, DP-31 <code>executeNextTxRevertsAfterExpiration</code> ✅ pass. It cannot run before the cooldown has passed or after it has expired. FV — transaction cannot be executed while paused: DP-18 <code>pausedBlocksExecuteNextTx</code> ✅ passes. It cannot run while PauseGuard is paused. FV — Only the transaction at the current txNonce executes: DP-28 <code>executeNextTxConsumesOnlyTheEntryAtTxNonce</code> ✅ passes. What executes is exactly the transaction that was queued. FV — transaction cannot be executed once vetoed: DP-32 <code>vetoedTransactionCannotExecute</code>, DP-33 <code>entryPassedByTxNonceNeverExecutes</code> ✅ pass. Once txNonce has moved past a proposal's nonce, it can never be executed. A proposal is vetoed by moving txNonce past it with <code>setTxNonce</code>.

Generalization 2 — Only TERM token holders and Term multisig holders may veto Delay modifier transactions, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard are held constant on the Delay and Roles Modifiers, and 3. the role targets, role scopes, and role assignments are also held constant on the Roles Modifier.

#	Statement	Evidence
G2.1	Veto power (<code>Delay.setTxNonce(uint256)</code>) is held exclusively by the owner of the DelayMod.	FV — setTxNonce is owner-only: DM-8 <code>atMostOneCallerPassesOnlyOwner</code> ✅ passes. <code>setTxNonce</code> is one of the Delay's ten <code>onlyOwner</code> functions: only the address recorded as the owner can call it successfully, and no two different callers can both get through from the same state.
G2.2	Only Modules or Owner can interact with Delay Modifier, except <code>executeNextTx</code> and <code>skipExpired</code> , which anyone can call.	Same as G1.6
G2.3	DelayMod only has one owner, which is the DelayOwnerSafe.	FV — one owner: DM-8 <code>atMostOneCallerPassesOnlyOwner</code> ✅ passes. The Delay has only one owner: at any moment, only one address can change the Delay's settings, and it is the address recorded as the owner. No second address can ever do it alongside it. On-chain — owner is DelayOwnerSafe: ⌚ pending migration. <code>owner()</code> on the Delay must return <code>0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3</code> . At block 26049693 it returns <code>0x405b47354CF06A25DE1DDb35EC65F03939E2e8D2</code> (old Roles).
G2.4	Only Modules or Multisig Holders can interact with DelayOwnerSafe, except fallback and receive, which anyone can call.	The same rules as G1.2, run against Safe v1.4.1 through the <code>Safe-v141-*</code> confs. FV — every entry point to the Safe is enumerated below: SE141-1 <code>safeEntryPointsAreAllAccountedFor</code> ✅ passes. Safe v1.4.1 has no write functions besides these sixteen entry points (the Prover checks <code>fallback</code> and <code>receive</code> as one entry), and any other call lands in <code>fallback</code> . These are G1.2's seventeen minus <code>requiredTxGas</code> , which Safe removed in v1.4.0. Grouped by who can call each one: <i>module gated entry points:</i> <code>execTransactionFromModule</code> <code>execTransactionFromModuleReturnData</code> <i>owner gated entry points:</i> <code>execTransaction</code> <code>approveHash</code> <i>Self calls — only modules and owners can reach these:</i> <code>enableModule</code> <code>disableModule</code> <code>addOwnerWithThreshold</code> <code>removeOwner</code> <code>swapOwner</code> <code>changeThreshold</code> <code>setGuard</code> <code>setFallbackHandler</code> <i>Non-interactions — these always revert, so nothing they do stands:</i>

#	Statement	Evidence
		• <code>setup</code> • <code>simulateAndRevert</code> <i>Open to anyone:</i> • <code>fallback</code> • <code>receive</code> FV — module-only entry points can make the Safe execute to any destination: <code>execTransactionFromModule</code>, <code>execTransactionFromModuleReturnData</code>. • SE141-3 <code>onlyEnabledModulesCanCallModuleExec</code> ✓ passes: only an enabled module can successfully call <code>execTransactionFromModule</code> or <code>execTransactionFromModuleReturnData</code>. • SE141-4 <code>enabledModuleCanMakeTheSafeAct</code> ✓ passes: an enabled module can call <code>execTransactionFromModule</code> and <code>execTransactionFromModuleReturnData</code> directly, with no owner signatures, and the Safe makes an outgoing call or <code>delegatecall</code>. FV — owner-only entry points can make the Safe execute to any destination: <code>execTransaction</code>, <code>approveHash</code>. • SE141-5 <code>execTransactionRunsOnlyAfterTheSignatureCheck</code> ✓ passes: <code>execTransaction</code> only runs once the signature threshold check has passed for that exact transaction. • SE141-12 <code>eachSignatureAcceptsANewApprovingOwner</code> ✓ passes: every signature the check accepts is a different owner who approved it. With SE141-5, the Safe acts through <code>execTransaction</code> only when as many different owners as the threshold have approved. • SE141-8 <code>onlyOwnersCanApproveHashes</code> ✓ passes: <code>approveHash</code> only records an owner's own approval, which then counts as that owner's signature in <code>execTransaction</code>. • SE141-6 <code>ownersCanMakeTheSafeAct</code> ✓ passes: once the threshold is passed, <code>execTransaction</code> succeeds and the Safe acts, for any threshold and any number of owners. FV — settings functions are only reached through the module-only or owner-only points: • SE141-7 <code>settingsOnlyChangeWhenTheSafeCallsItself</code> ✓ passes: these only change a setting when the caller is the Safe itself. • SC141-4 <code>settingsOnlyChangeThroughAModuleOrOwners</code> ✓ passes: over every write function, <code>fallback</code> included, a setting only changes through an enabled module's <code>execTransactionFromModule</code> or <code>execTransactionFromModuleReturnData</code>, or through <code>execTransaction</code>. A <code>fallback</code> handler is checked as if it were the Safe itself, so it cannot reach these functions even then. • SC141-1 <code>moduleCanChangeSettings</code>, SC141-2 <code>moduleCanChangeSettingsWithReturnData</code> ✓ pass: for each function above, a module's <code>execTransactionFromModule</code>, and likewise its <code>execTransactionFromModuleReturnData</code>, addressed to the Safe with a call to that function, succeeds and changes a setting. • SC141-3 <code>ownersCanChangeSettings</code> ✓ passes: the same through <code>execTransaction</code>, once the threshold is passed. FV — setup is not a callable function after deploy: SE141-9 <code>setupAlwaysRevertsAfterSetup</code> ✓ passes. Once the Safe is set up, <code>setup</code> always reverts, so nobody can re-run it to replace the owners, threshold, modules or handler.

#	Statement	Evidence
		FV — simulateAndRevert always reverts: SE141-11 <code>simulateAndRevertAlwaysReverts</code> ✅ passes. Anyone can call it, and it does make a delegatecall, but it always reverts afterwards, so nothing it does stands. FV — Any address can call any unsupported Safe function to initiate fallback, which can only make the Safe execute to Fallback Handler address if set to nonzero address: • SE141-14 <code>fallbackOnlyCallsItsHandler</code> ✅ passes: with a handler set, <code>fallback</code> only makes a plain call, with no ETH, to the handler. It never delegatecalls, so the handler's code runs as its own contract, not as the Safe, and cannot write the Safe's storage. Assumes the handler is not the Safe itself: v1.4.1's <code>setFallbackHandler</code> and <code>setup</code> refuse that (GS400), but a delegatecall run through <code>execTransaction</code> or a module can still write the handler slot directly. That case is covered by SE141-16. • SE141-15 <code>fallbackHandlerCallsComeFromTheHandler</code> ✅ passes: once <code>fallback</code> calls its handler, every call the handler makes uses <code>msg.sender = handler</code>. Assumes the handler is not the Safe itself; that case is SE141-16. • SE141-16 <code>selfHandlerFallbackChangesNothing</code> ✅ passes: with the Safe set as its own handler, <code>fallback</code> leaves the Safe's storage exactly as it was, and when it succeeds the Safe called nothing but itself, sent no ETH and made no delegatecall. • SE141-13 <code>fallbackMakesNoCallWithoutAHandler</code> ✅ passes: with no handler set (the zero address), <code>fallback</code> makes no call or delegatecall at all. FV — Any address can trigger receive by sending Eth without calldata and it does nothing but emits SafeReceived: • SE141-17 <code>receiveOnlyEmitsSafeReceived</code> ✅ passes: whenever <code>receive</code> runs, it does nothing beyond the event. It emits exactly one log, <code>SafeReceived</code>, naming the caller as sender. It makes no call of any kind (call, delegatecall, staticcall or callcode) and writes no storage. • SE141-18 <code>receiveCanRun</code> ✅ passes: some call to the Safe succeeds and emits <code>SafeReceived</code>, so SE141-17 is not vacuous.
G2.5	DelayOwnerSafe fallback handler is <code>address(0)</code> .	On-chain: ✅ the fallback handler slot (0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5, <code>FallbackManager.sol:14</code> in Safe v1.4.1) on the DelayOwnerSafe (0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3) is address(0) at block 26080124.
G2.6	DelayOwnerSafe owners are Term multisig holders.	On-chain — eleven keys, any five act: ✅ at block 26084873, the DelayOwnerSafe (0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3) runs Safe v1.4.1 (VERSION), has threshold 5 (<code>getThreshold</code>) and these 11 owners (<code>getOwners</code>), each an externally owned account with no contract code: • 0xeee661edcFE634Dc0e29D62C26AfD62c0843b817 • 0xF6A745f9B38FFcd17Ee8909AC89151D788F95282 • 0xC43d527E3544A3d199Ce28E87994384D89d90e6E • 0x8EF485fD38b7B29a827938C9F00300eaBf8E1710 • 0xbFfcAdCd5549cC378693108Bcd4435776A6fa795 • 0xB680373c50E9E877DA7dCF9efcc0dFB234452ad3 • 0x6eb0c274EC4B1d51152e45BAED22f883B2A3Bc60 • 0xE82183cfAE1C24f044315c318FD97bE7e47b31D2 • 0xc1047a4D9f6071B55585523F7F659F39A1fBA74b • 0xDB97c377F23a9Dc54c3D6F9df7bC0ca4Cf7D5A81 • 0x89562FC5AEF155481aE0C7dde1300110B4dF2F1D FV — the owner set only changes through the Safe itself: SE141-7

#	Statement	Evidence
		<code>settingsOnlyChangeWhenTheSafeCallsItself</code>, SC141-4 <code>settingsOnlyChangeThroughAModuleOrOwners</code>  pass. <code>addOwnerWithThreshold</code>, <code>removeOwner</code>, <code>swapOwner</code> and <code>changeThreshold</code> only take effect when the Safe calls itself: through <code>execTransaction</code>, with 5 of the 11 owners (SE141-5, SE141-12), or through its only module, the Roles Modifier (G2.7), which can only send <code>Delay.setTxNonce(uint256)</code> (G2.12). SE141-9 <code>setupAlwaysRevertsAfterSetup</code>  passes: <code>setup</code> cannot be re-run to replace them. Who holds the eleven keys is not visible on chain. That they are Term multisig holders is a deployment fact.
G2.7	DelayOwnerSafe only has one enabled module, the Roles Modifier	On-chain — the DelayOwnerSafe's module list contains only the Roles Modifier:  asked for all of its modules, the DelayOwnerSafe (<code>0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3</code>) returns just one, the Roles Modifier (<code>0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3</code>), and reports that the list ends there (<code>getModulesPaginated</code>, block 26080124). On-chain — the DelayOwnerSafe's storage holds only the Roles Modifier as a module:  read directly from storage rather than through the Safe's own functions, the module list starts at the Roles Modifier and the Roles Modifier points back to the start, so it is the only entry (<code>modules</code>, storage slot 1, block 26080124).
G2.8	Only Modules with assignedRoles or Owner can interact with Roles Modifier.	FV — every entry point to the Roles Modifier is enumerated below: RI-14 <code>rolesWriteFunctionsAreTheKnownTwentyFive</code>  passes. The Roles Modifier has no <code>fallback</code> or <code>receive</code> and no write functions besides these twenty-five. Grouped by who can call each one: <i>entry points gated to enabled modules with assigned roles:</i> • <code>execTransactionFromModule</code> (the caller's default role) • <code>execTransactionFromModuleReturnData</code> (the caller's default role) • <code>execTransactionWithRole</code> (the role the call names) • <code>execTransactionWithRoleReturnData</code> (the role the call names) <i>owner gated entry points — the settings functions:</i> • <code>setMultisend</code> • <code>allowTarget</code> • <code>revokeTarget</code> • <code>scopeTarget</code> • <code>scopeAllowFunction</code> • <code>scopeRevokeFunction</code> • <code>scopeFunction</code> • <code>scopeFunctionExecutionOptions</code> • <code>scopeParameter</code> • <code>scopeParameterAsOneOf</code> • <code>unscopeParameter</code> • <code>assignRoles</code> • <code>setDefaultRole</code> • <code>setAvatar</code> • <code>setTarget</code> • <code>enableModule</code> • <code>disableModule</code> • <code>setGuard</code> • <code>transferOwnership</code> • <code>renounceOwnership</code>

#	Statement	Evidence
		<i>Non-interactions — this always reverts, so nothing it does stands:</i> • <code>setUp</code> FV — execution functions succeed only if caller address is an enabled module AND has been assigned the permissioned role <code>execTransactionFromModule</code>, <code>execTransactionFromModuleReturnData</code>, <code>execTransactionWithRole</code>, <code>execTransactionWithRoleReturnData</code>. • RI-7 <code>onlyEnabledModulesCanExec</code> ✓ passes: only enabled modules can call exec functions • RI-8 <code>moduleWithoutDefaultRoleCannotExecFromModule</code> ✓ passes: an enabled module that is not a member of its default role cannot execute through <code>execTransactionFromModule</code> or <code>execTransactionFromModuleReturnData</code>. • RI-9 <code>moduleWithoutRoleCannotExecTransactionWithRole</code>, RI-10 <code>moduleWithoutRoleCannotExecTransactionWithRoleReturnData</code> ✓ pass: an enabled module that is not a member of the role the call names cannot execute through <code>execTransactionWithRole</code> or <code>execTransactionWithRoleReturnData</code>. With RI-8, a module with no assigned role can execute nothing through any of the four. • RI-11 <code>roleMemberModuleCanExecFromModule</code>, RI-12 <code>roleMemberModuleCanExecTransactionWithRole</code>, RI-13 <code>roleMemberModuleCanExecTransactionWithRoleReturnData</code> ✓ pass: an enabled module that is a member of the role the call runs under can successfully call each of the four, so the restriction is not achieved by nothing working. FV — owner-only settings: RI-2 <code>rolesConfigOnlyChangesThroughOwner</code>, RI-3 <code>ownerOnlyChangesThroughOwnableTransfer</code>, RI-4 <code>ownerCanStillReconfigure</code>, RI-5 <code>onlyOwnerCanCallRolesSettings</code>, RI-6 <code>ownerCanCallEachRolesSetting</code> ✓ pass. Each of the twenty settings functions only succeeds for the owner (RI-5), and the owner can successfully call each one (RI-6); FV — setup is not a callable function after deploy: RI-1 <code>setUpAlwaysRevertsAfterDeployment</code> ✓ passes. Once the module list is set up, <code>setUp</code> always reverts, so nobody can re-run it to replace the owner or the module list.
G2.9	Only enabled modules of the Roles Modifier can execute transactions through it. Roles Modifier has no owner, since its owner value is <code>address(0)</code> .	FV — only enabled modules can execute: RI-7 <code>onlyEnabledModulesCanExec</code> ✓ passes. On all four execution entry points (<code>execTransactionFromModule</code>, <code>execTransactionFromModuleReturnData</code>, <code>execTransactionWithRole</code>, <code>execTransactionWithRoleReturnData</code>), a call that succeeds came from an address in the module list. No owner: same as G3.RolesModifier.2b.
G2.10	The Governor is the only enabled module of the Roles Modifier with an assignedRole.	On-chain — the Roles Modifier's module list contains only the Governor: ✓ asked for all of its modules, the Roles Modifier (<code>0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3</code>) returns just one, the Governor (<code>0x2B715634134220ffeEE9458b4e34E41A41418607</code>), and reports that the list ends there (<code>getModulesPaginated</code>, block 26080168). On-chain — the Roles Modifier's storage holds only the Governor as a module: ✓ read directly from storage rather than through the Roles Modifier's own functions, the module list starts at the Governor and the Governor points back to the start, so it is the only entry (<code>modules</code>, storage slot 104, block 26080168).

#	Statement	Evidence
		On-chain — the Governor holds a role: ✓ the Governor is recorded as a member of role 1 (<code>roles[1].members</code> , storage slot 107, block 26080168). On-chain — no other module has ever been added or given a role: ✓ the Roles Modifier's complete event history, from its creation at block 26042039 to block 26080168, has one <code>EnabledModule</code> , for the Governor, and no <code>DisabledModule</code> . It has one <code>AssignRoles</code> , also for the Governor.
G2.11	The Governor is assigned only role 1, its defaultRole.	On-chain — the Governor's default role is 1: ✓ <code>defaultRoles</code> on the Roles Modifier (<code>0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3</code>) returns 1 for the Governor (<code>0x2B715634134220ffeEE9458b4e34E41A41418607</code>) at block 26080168. On-chain — the Governor is a member of role 1: ✓ the Governor is recorded as a member of role 1 (<code>roles[1].members</code> , storage slot 107, block 26080168). On-chain — the Governor has only ever been given role 1: ✓ a module's roles can only change through <code>assignRoles</code> (<code>Roles.sol:290</code>), which always emits <code>AssignRoles</code> . The Roles Modifier's complete event history, from its creation at block 26042039 to block 26080168, has exactly one <code>AssignRoles</code> : the Governor, role 1, member (tx <code>0x1d4ec5029b9c66cfff2c92bc8be16196bba4d4484caaa74488bdcc61616fb186a</code>). It also has exactly one <code>SetDefaultRole</code> : the Governor, role 1 (tx <code>0xf01e95980c86dc59667c78094ba1f7a65c744b5214029ef70e721ec8c340a928</code>).
G2.12	Role 1 and the SetTxNonceGuard together restrict the Governor to only vetos.	FV — role 1 alone restricts member module executions to vetos (Delay.setTxNonce(uint256)), except through MultiSend: RC-1 <code>roleConfigLimitsExecTransactionWithRoleToDelaySetTxNonce</code> , RC-2 <code>roleConfigLimitsExecTransactionFromModuleToDelaySetTxNonce</code> , RC-3 <code>roleConfigLimitsExecTransactionWithRoleReturnDataToDelaySetTxNonce</code> , RC-4 <code>roleConfigLimitsExecTransactionFromModuleReturnDataToDelaySetTxNonce</code> ✓ pass. With no <code>setTxNonceGuard</code> , role 1's scope configuration admits only <code>setTxNonce(uint256)</code> on the Delay, with zero value and as a plain <code>Call</code> , on all four entry points, provided the destination is not the <code>MultiSend</code> address. • RC-5 <code>nonMemberExecTransactionWithRoleAlwaysReverts</code> ✓ passes: a caller that is not a member of the role it names can execute nothing at all. Stated on <code>execTransactionWithRole</code> . • RC-7 <code>withoutSetTxNonceGuardMultisendTargetEscapesRoleConfig</code> ✓ passes: without the guard, a transaction addressed to the <code>MultiSend</code> address completes although it is not the Delay. This is the gap the guard closes (RS-6). FV — setTxNonceGuard alone restricts all Roles Modifier executions to vetos <code>Delay.setTxNonce(uint256)</code> : RS-1 <code>setTxNonceGuardLimitsExecTransactionWithRoleToDelaySetTxNonce</code> , RS-2 <code>setTxNonceGuardLimitsExecTransactionFromModuleToDelaySetTxNonce</code> , RS-3 <code>setTxNonceGuardLimitsExecTransactionWithRoleReturnDataToDelaySetTxNonce</code> , RS-4 <code>setTxNonceGuardLimitsExecTransactionFromModuleReturnDataToDelaySetTxNonce</code> ✓ pass. With <code>SetTxNonceGuard</code> installed, every execution on

#	Statement	Evidence
		all four entry points is <code>setTxNonce(uint256)</code> on the Delay, with zero value and as a plain <code>Call</code>, for any caller, any role and any Roles configuration. • RS-5 <code>setTxNonceGuardLimitsToDelaySetTxNonceUnderMaximallyPermissiveRoles</code> passes: the same holds under the worst-case configuration, blanket <code>Clearance.Target</code> with <code>ExecutionOptions.Both</code>. • RS-6 <code>setTxNonceGuardRejectsMultisendTarget</code> passes: a transaction addressed to the <code>MultiSend</code> address always reverts. • RS-7 <code>withoutSetTxNonceGuardPermissiveRolesAllowNonDelayCall</code> passes: with no guard and the same permissive configuration, a call to something other than the Delay succeeds, so it is the guard that imposes the restriction. FV — With role 1 and SetTxNonceGuard applied together, the Governor can only veto: RG-2 <code>governorExecTransactionWithRoleLimitedToDelaySetTxNonce</code>, RG-3 <code>governorExecTransactionFromModuleLimitedToDelaySetTxNonce</code>, RG-4 <code>governorExecTransactionWithRoleReturnDataLimitedToDelaySetTxNonce</code>, RG-5 <code>governorExecTransactionFromModuleReturnDataLimitedToDelaySetTxNonce</code> pass. Every call the Governor completes through the Roles Modifier is <code>setTxNonce(uint256)</code> on the Delay, with zero value and as a plain <code>Call</code>, on all four execution entry points: <code>execTransactionWithRole</code>, <code>execTransactionFromModule</code>, <code>execTransactionWithRoleReturnData</code> and <code>execTransactionFromModuleReturnData</code>. Assumes <code>SetTxNonceGuard</code> is installed and pointed at the Delay, and the Governor is an enabled module, a member of role 1 only, with default role 1 (G2.11). FV — Governor successfully vetoes: RG-6 <code>governorCanStillCallDelaySetTxNonce</code> passes. Under the same setup, the Governor's <code>setTxNonce</code> call really does go through, so the restriction is not achieved by nothing working.
G2.13	Only TERM Token holders can create a Governor proposal to veto, and once it passes vote, successfully execute.	FV — only TERM holders can propose: GV-1 <code>holderAboveThresholdCanProposeVeto</code> passes: any address with at least 1,000 TERM of votes at <code>clock() - 1</code> can propose the veto, the one-action proposal calling the Roles Modifier with <code>execTransactionWithRole(Delay, 0, setTxNonce(n), Call, 1, true)</code>, for any <code>n</code> and any description of up to 1,024 bytes that propose accepts from it, unless that exact proposal already exists. The proposal records the caller as its proposer, and its vote opens at once and runs for 22 hours. GP-1 <code>holderBelowThresholdCannotPropose</code> passes: for any caller whose votes at <code>clock() - 1</code> are below the threshold, and for any targets, values, calldatas and description, <code>propose</code> reverts (<code>GovernorInsufficientProposerVotes</code>), so no proposal of any kind is created. The Governor proposing to itself through <code>relay</code> is one such caller, with its own votes. GP-1 states the threshold against the Governor's own vote count, and GV-4 <code>governorVotesAreTermPastVotes</code> passes shows that count is TERM's: for any account and any past timepoint, the Governor's <code>getVotes</code> equals TERM's <code>getPastVotes</code>. Together, exactly the TERM holders at or above the threshold can propose. FV — only TERM holders' votes count: anyone can cast a vote, but a vote adds exactly the voter's votes at the proposal's snapshot, so an account without TERM votes adds nothing, and nothing else moves a tally. GC-1 <code>directVoteCountsOnlyTheVotersVotes</code> passes: through <code>castVote</code>, <code>castVoteWithReason</code> and <code>castVoteWithReasonAndParams</code>, a proposal's

#	Statement	Evidence
		tally grows by exactly the caller's votes at its snapshot if the caller voted on it, and does not move otherwise. GC-2 voteBySigCountsOnlyTheVotersVotes and GC-3 voteWithReasonAndParamsBySigCountsOnlyTheVotersVotes  pass: the same through castVoteBySig and castVoteWithReasonAndParamsBySig, counting the voter named in the call, whoever submits it. GC-4 nothingButAVoteMovesATally  passes: every other write function, execute and relay included, leaves every tally where it was; queue never succeeds (GS-4). GC-5 termHolderVoteCounts  passes: a caller with votes at the snapshot can vote, and the tally moves, so the rules are not satisfied by nothing working. FV — once it passes, the veto proposal executes and carries through to the Delay Modifier: Once the veto's state is Succeeded, anyone's execute succeeds and the Delay's txNonce becomes n, for any n the Delay accepts ($txNonce < n \leq queueNonce$). This follows from two rules that meet at the Roles Modifier's module call on the DelayOwnerSafe: • GV-2 passedVetoReachesTheDelayOwnerSafe  passes: once the veto's state is Succeeded, anyone's execute succeeds, and the Roles Modifier forwards the veto to the DelayOwnerSafe as exactly one module call, <code>execTransactionFromModule(Delay, 0, setTxNonce(n), Call)</code>, for any n, provided that call returns true. Assumes the deployed wiring: SetTxNonceGuard is the Roles Modifier's guard and points at the Delay, role 1 is scoped to Delay.setTxNonce, and the Governor is an enabled module in role 1 (G2.10, G2.11). A recording stand-in takes the DelayOwnerSafe's place. • GV-3 delayOwnerSafeLandsTheVeto  passes: on Safe v1.4.1, an enabled module's <code>execTransactionFromModule(Delay, 0, setTxNonce(n), Call)</code> returns true and sets the Delay's txNonce to n, for any n the Delay accepts, when the Safe owns the Delay (G2.3). On chain that module is the Roles Modifier (G2.7).

Generalization 3 — The following: 1. the owners, fallbackHandler, modules, guard for the ProposerSafe, OwnerlessSafe, and DelayOwnerSafe, and 2. the owner, target, modules, guard for the Delay Modifier, and 3. the owner, target, modules, guard, role targets, role scopes, and role assignments for the Roles Modifier, cannot be changed other than by the Term multisig holders on the ownerless safe, proposer safe, or DelayModOwner safe.

Ownerless Safe

Settings functions (GnosisSafe v1.3.0; only the Safe itself can call them):

- enableModule(address)
- disableModule(address,address)
- addOwnerWithThreshold(address,uint256)
- removeOwner(address,address,uint256)
- swapOwner(address,address,address)
- changeThreshold(uint256)
- setGuard(address)
- setFallbackHandler(address)
- setup(address[],uint256,address,bytes,address,address,uint256,address) , one-time initializer: always reverts once the Safe is set up

Conclusion: The modules, owners, guard, fallbackHandler of the Ownerless Safe cannot be changed other than by its signers or its only enabled module, the Delay Modifier. The Delay Modifier can only execute proposals queued by the signers of the Proposer Safe. Additional proposal modules can only be added by the signers on the DelayOwnerSafe.

Ownerless Safe · Table 1 - Only Ownerless Safe's owners and the Delay Modifier, the only enabled module, can modify the Ownerless Safe's settings.

#	Statement	Evidence
G3.OwnerlessSafe.1	Only Modules or Owners (Signers) can interact with OwnerlessSafe, except fallback and receive, which anyone can call. The settings functions of the OwnerlessSafe can only be executed by Owners or enabled modules.	Same as G1.2.
G3.OwnerlessSafe.2	OwnerlessSafe fallback handler is address(0) .	Same as G1.3.
G3.OwnerlessSafe.3	OwnerlessSafe owners are Term multisig holders.	Same as G1.4.
G3.OwnerlessSafe.4	Delay Module is the only enabled module on OwnerlessSafe.	Same as G1.5.
G3.OwnerlessSafe.5	Only Modules or Owner can interact with Delay Modifier, except executeNextTx and skipExpired, which anyone can call.	Same as G1.6.

After G3.OwnerlessSafe.5 the branch splits in two at the Ownerless Safe's only enabled module the Delay Modifier: the Delay Modifier's modules, and the Delay Modifier's owner.

Ownerless Safe · Table 2a — Conclusion: The only enabled module of the Delay Modifier, the Proposer Safe, can propose transactions. Each one waits out the cooldown and can be vetoed.

#	Statement	Evidence
G3.OwnerlessSafe.6a	ProposerSafe is the only enabled module on the Delay Mod	Same as G1.8.
G3.OwnerlessSafe.7a	Only Modules or Owners (Signers) can interact with ProposerSafe, except fallback and receive, which anyone can call.	Same as G1.9.
G3.OwnerlessSafe.8a	ProposerSafe fallback handler is address(0).	Same as G1.10.
G3.OwnerlessSafe.9a	ProposerSafe owners are Term multisig holders.	Same as G1.11.
G3.OwnerlessSafe.10a	ProposerSafe has no enabled modules.	Same as G1.12.
G3.OwnerlessSafe.11a	ProposerSafe can propose a transaction to Delay Modifier that is executed on the Ownerless Safe once the cooldown has passed, if it hasn't expired, the system isn't paused, and it hasn't been vetoed.	Same as G1.13.

Ownerless Safe · Table 2b — Conclusion: The owner of the Delay Modifier, the DelayOwnerSafe, can call the Delay Modifier's settings functions. It can execute these functions only through its signers, not thru its only enabled module, the Governor.

#	Statement	Evidence
G3.OwnerlessSafe.6b	DelayMod only has one owner, which is the DelayOwnerSafe.	Same as G2.3.
G3.OwnerlessSafe.7b	Only Modules or Owners (Signers) can interact with DelayOwnerSafe, except fallback and receive, which anyone can call.	Same as G2.4.
G3.OwnerlessSafe.8b	DelayOwnerSafe fallback handler is <code>address(0)</code> .	Same as G2.5.
G3.OwnerlessSafe.9b	DelayOwnerSafe owners are Term multisig holders.	Same as G2.6.
G3.OwnerlessSafe.10b	DelayOwnerSafe only has one enabled module, the Roles Modifier	Same as G2.7.
G3.OwnerlessSafe.11b	Only Modules with assignedRoles or Owners can interact with Roles Modifier.	Same as G2.8.
G3.OwnerlessSafe.12b	Roles Modifier has no owner, since its owner value is address(0).	Same as G3.RolesModifier.2b.

#	Statement	Evidence
G3.OwnerlessSafe.13b	The Governor is the only module of the Roles Modifier with an assignedRole.	Same as G2.10.
G3.OwnerlessSafe.14b	The Governor is assigned only role 1, its defaultRole.	Same as G2.11.
G3.OwnerlessSafe.15b	Role 1 and the SetTxNonceGuard together restrict the Governor to only vetos.	Same as G2.12.

Delay Modifier

Settings functions (zodiac-modifier-delay v1.0.1; `onlyOwner`):

- `setTxCooldown(uint256)`
- `setTxExpiration(uint256)`
- `setAvatar(address)`
- `setTarget(address)`
- `enableModule(address)`
- `disableModule(address,address)`
- `setGuard(address)`
- `transferOwnership(address)`
- `renounceOwnership()`
- `setUp(bytes)` , one-time initializer: always reverts after deployment

Conclusion: The modules, owner, guard, avatar, target, cooldown, expiration of the Delay Modifier cannot be changed other than by its owner, the DelayOwnerSafe. Only the signers on the DelayOwnerSafe can execute Delay settings functions. The DelayOwnerSafe's only enabled module, the Roles Modifier, cannot execute DelayOwnerSafe settings functions. Modifying the roles and members of the Roles Modifier cannot be performed by any caller. Additional modules can only be added to DelayOwnerSafe by its own signers.

Delay Modifier · Table 1 - Only the owner of the Delay Modifier, the DelayOwnerSafe, can call the Delay Modifier's settings functions. The DelayOwnerSafe, can only receive interactions from its signers/owners, or its only enabled module the Roles Modifier.

#	Statement	Evidence
G3.DelayModifier.1	Only Modules or Owner can interact with Delay Modifier, except <code>executeNextTx</code> and <code>skipExpired</code> , which anyone can call. The settings functions are owner-only: only the Owner can successfully execute them.	Same as G1.6.
G3.DelayModifier.2	DelayMod only has one owner, which is the DelayOwnerSafe.	Same as G2.3.
G3.DelayModifier.3	Only Modules or Owners can interact with DelayOwnerSafe.	Same as G2.4.

#	Statement	Evidence
G3.DelayModifier.4	DelayOwnerSafe fallback handler is <code>address(0)</code> .	Same as G2.5.
G3.DelayModifier.5	DelayOwnerSafe owners are Term multisig holders.	Same as G2.6.
G3.DelayModifier.6	DelayOwnerSafe only has one module, the Roles Modifier	Same as G2.7.
G3.DelayModifier.7	Only Modules with assignedRoles or Owners can interact with Roles Modifier.	Same as G2.8.

After G3.DelayModifier.7 the branch splits in two at the DelayOwnerSafe's only enabled module the Roles Modifier: the Roles Modifier's modules, and the Roles Modifier's owner.

Delay Modifier · Table 2a — The Roles Modifier, the only enabled module of the DelayOwnerSafe, can only execute veto transactions `Delay.setTxNonce(uint256)` from the Governor contract.

#	Statement	Evidence
G3.DelayModifier.8a	The Governor is the only enabled module of the Roles Modifier with an assignedRole.	Same as G2.10.
G3.DelayModifier.9a	The Governor is assigned only role 1, its defaultRole.	Same as G2.11.
G3.DelayModifier.10a	Role 1 and the SetTxNonceGuard together restrict the Governor to only vetos.	Same as G2.12.

Delay Modifier · Table 2b — The Roles Modifier, the only enabled module of the DelayOwnerSafe, has no owner, since its owner value is `address(0)`. The Roles Modifier's settings cannot be changed by anyone.

#	Statement	Evidence
G3.DelayModifier.8b	Roles Modifier has no owner, since its owner value is <code>address(0)</code> .	Same as G3.RolesModifier.2b.

Proposer Safe

Settings functions (GnosisSafe v1.3.0; only the Safe itself can call them):

- `enableModule(address)`
- `disableModule(address,address)`
- `addOwnerWithThreshold(address,uint256)`
- `removeOwner(address,address,uint256)`
- `swapOwner(address,address,address)`
- `changeThreshold(uint256)`
- `setGuard(address)`
- `setFallbackHandler(address)`

- `setup(address[],uint256,address,bytes,address,address,uint256,address)` , one-time initializer: always reverts once the Safe is set up

Conclusion: The modules, owners, guard, fallbackHandler of the Proposer Safe cannot be changed other than by Term Multisig owners.

Proposer Safe · Table 1 — Conclusion: The settings functions of the Proposer Safe can only be executed by its signers since it has no enabled modules.

#	Statement	Evidence
G3.ProposerSafe.1	Only Modules or Owners (Signers) can interact with ProposerSafe, except fallback and receive, which anyone can call. The settings functions of the ProposerSafe can only be executed by Owners or enabled modules.	Same as G1.9.
G3.ProposerSafe.2	ProposerSafe fallback handler is address(0).	Same as G1.10.
G3.ProposerSafe.3	ProposerSafe owners are Term multisig holders.	Same as G1.11.
G3.ProposerSafe.4	ProposerSafe has no enabled modules.	Same as G1.12.

DelayOwnerSafe

Settings functions (Safe v1.4.1; only the Safe itself can call them):

- `enableModule(address)`
- `disableModule(address,address)`
- `addOwnerWithThreshold(address,uint256)`
- `removeOwner(address,address,uint256)`
- `swapOwner(address,address,address)`
- `changeThreshold(uint256)`
- `setGuard(address)`
- `setFallbackHandler(address)`
- `setup(address[],uint256,address,bytes,address,address,uint256,address)` , one-time initializer: always reverts once the Safe is set up

Conclusion: The modules, owners, guard, fallbackHandler of the DelayOwnerSafe cannot be changed other than by its signers. The DelayOwnerSafe's only enabled module, the Roles Modifier, cannot execute DelayOwnerSafe settings functions. Modifying the roles and members of the Roles Modifier cannot be done by any caller. Additional modules can only be added to DelayOwnerSafe by its own signers.

DelayOwnerSafe · Table 1 — The DelayOwnerSafe's settings can only be changed by its owners or through the its only enabled module, the Roles Modifier.

#	Statement	Evidence
G3. DelayOwnerSafe.1	Only Modules or Owners (Signers) can interact with DelayOwnerSafe, except fallback and receive, which anyone can call. The settings functions of the DelayOwnerSafe can only be executed by Owners or enabled modules.	Same as G2.4.
G3. DelayOwnerSafe.2	DelayOwnerSafe fallback handler is <code>address(0)</code> .	Same as G2.5.
G3. DelayOwnerSafe.3	DelayOwnerSafe owners are Term multisig holders.	Same as G2.6.
G3. DelayOwnerSafe.4	DelayOwnerSafe only has one enabled module, the Roles Modifier	Same as G2.7.
G3. DelayOwnerSafe.5	Only Modules with assignedRoles or Owners can interact with Roles Modifier.	Same as G2.8.

After G3.DelayOwnerSafe.5 the branch splits in two at the DelayOwnerSafe's only module, the Roles Modifier: the Roles Modifier's modules, and the Roles Modifier's owner.

DelayOwnerSafe · Table 2a — The Roles Modifier, the only enabled module of the DelayOwnerSafe, can only execute veto transactions `Delay.setTxNonce(uint256)` from the Governor contract.

#	Statement	Evidence
G3. DelayOwnerSafe.6a	The Governor is the only enabled module of the Roles Modifier with an assignedRole.	Same as G2.10.
G3. DelayOwnerSafe.7a	The Governor is assigned only role 1, its defaultRole.	Same as G2.11.
G3. DelayOwnerSafe.8a	Role 1 and the SetTxNonceGuard together restrict the Governor to only vetos.	Same as G2.12.

DelayOwnerSafe · Table 2b — The Roles Modifier, the only enabled module of the DelayOwnerSafe, has no owner, since its owner value is address(0). The Roles Modifier's settings cannot be changed by anyone.

#	Statement	Evidence
G3. DelayOwnerSafe.6b	Roles Modifier has no owner, since its owner value is address(0).	Same as G3.RolesModifier.2b.

Roles Modifier

Settings functions (Roles v1.0.0; `onlyOwner`):

- `setMultisend(address)`
- `allowTarget(uint16,address,ExecutionOptions)`
- `revokeTarget(uint16,address)`
- `scopeTarget(uint16,address)`
- `scopeAllowFunction(uint16,address,bytes4,ExecutionOptions)`
- `scopeRevokeFunction(uint16,address,bytes4)`
- `scopeFunction(uint16,address,bytes4,bool[],ParameterType[],Comparison[],bytes[],ExecutionOptions)`
- `scopeFunctionExecutionOptions(uint16,address,bytes4,ExecutionOptions)`
- `scopeParameter(uint16,address,bytes4,uint256,ParameterType,Comparison,bytes)`
- `scopeParameterAsOneOf(uint16,address,bytes4,uint256,ParameterType,bytes[])`
- `unscopeParameter(uint16,address,bytes4,uint8)`
- `assignRoles(address,uint16[],bool[])`
- `setDefaultRole(address,uint16)`
- `setAvatar(address)`
- `setTarget(address)`
- `enableModule(address)`
- `disableModule(address,address)`
- `setGuard(address)`
- `transferOwnership(address)`
- `renounceOwnership()`
- `setUp(bytes)` , one-time initializer: reverts once initialized

Conclusion: The owner, target, modules, guard, role targets, role scopes, and role assignments on the Roles Modifier cannot be changed by any caller.

Roles Modifier · Table 1 — Only the Roles Modifier's owner or enabled modules may interact with the Roles Modifier.

#	Statement	Evidence
G3.RolesModifier.1	Only Modules with assignedRoles or Owners can interact with Roles Modifier. The settings functions are owner-only: only the Owner can successfully execute them.	Same as G2.8.

After G3.RolesModifier.1 the branch splits in two: the Roles Modifier's modules, and the Roles Modifier's owner.

Roles Modifier · Table 2a — The Roles Modifier's only enabled module, the Governor Contract, can only execute veto transactions `Delay.setTxNonce(uint256)` .

#	Statement	Evidence
G3.RolesModifier.2a	The Governor is the only module of the Roles Modifier with an assignedRole.	Same as G2.10.
G3.RolesModifier.3a	The Governor is assigned only role 1, its defaultRole.	Same as G2.11.
G3.RolesModifier.4a	Role 1 and the SetTxNonceGuard together restrict the Governor to only vetos.	Same as G2.12.

Roles Modifier · Table 2b — The Roles Modifier has no owner, since its owner value is address(0). The Roles Modifier's settings cannot be changed by anyone.

#	Statement	Evidence
G3.RolesModifier.2b	Roles Modifier has no owner, since its owner value is address(0).	FV — settings functions are unreachable when owner is address(0): RI-15 noOwnerSettingsAlwaysRevert  passes: with the owner at address(0), each of the twenty settings functions reverts for every caller, since no transaction comes from address(0). RI-16 noOwnerStaysNoOwner  passes: no function can give the Roles Modifier an owner back (setUp is covered by RI-1, G2.8). RI-17 renounceOwnershipLeavesNoOwner  passes: the owner's renounceOwnership() succeeds and leaves the Roles Modifier with no owner, so RI-15 and RI-16 are not about an unreachable state. On-chain — owner is address(0): ⌚ pending. owner() on the Roles Modifier (0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3) must return 0x00, which the Ownerless Safe sets by calling renounceOwnership(). At block 26083996 it returns the Ownerless Safe (0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03), and _owner (slot 51, OwnableUpgradeable) holds the same address.

Governor

Settings functions: none. **Conclusion:** There are no settings functions on the Governor contract.

Governor · Table 1 — Nobody can change the Governor's settings.

#	Statement	Evidence
G3.Governor.1	Governor does not have settings functions.	FV — every write function of the Governor is enumerated below, and none is a setter: GS-1 governorWriteFunctionsAreTheKnownFourteen  passes. TermFinanceGovernor builds on OpenZeppelin's Governor, GovernorVotes and GovernorCountingSimple only, not GovernorSettings, GovernorVotesQuorumFraction or GovernorTimelockControl, the extensions that add setters. It has no write functions besides these fourteen: <i>proposal lifecycle:</i> - propose - execute - cancel <i>voting:</i> - castVote - castVoteWithReason - castVoteWithReasonAndParams - castVoteBySig - castVoteWithReasonAndParamsBySig <i>governance only — callable only by the Governor itself, as an action of an executed proposal:</i> - relay, which makes one call out and writes nothing <i>token and ETH receipt — each writes nothing:</i> - onERC721Received - onERC1155Received - onERC1155BatchReceived

#	Statement	Evidence
		• <code>receive</code> <i>Non-interactions — this always reverts, so nothing it does stands:</i> • <code>queue</code> FV — <code>queue</code> always reverts: GS-4 <code>queueAlwaysReverts</code>  passes. The Governor has no timelock, so it never overrides OpenZeppelin's <code>_queueOperations</code>, which returns 0, and <code>queue</code> then reverts with <code>GovernorQueueNotImplemented</code>. A proposal that passes goes straight to <code>execute</code>. FV — no function changes a setting: GS-2 <code>governorSettingsNeverChange</code>  passes. For every write function except <code>queue</code> (GS-4), including <code>relay</code> called by the Governor as a proposal action, any caller and any arguments, every setting reads the same afterwards: <code>name</code>, <code>version</code>, <code>COUNTING_MODE</code>, the executor, <code>token</code>, <code>votingDelay</code>, <code>votingPeriod</code>, <code>proposalThreshold</code>, <code>quorumNumerator</code>, <code>quorumDenominator</code> and <code>proposalNeedsQueuing</code>. This holds for a function added later too. FV — the settings are compiled in: GS-3 <code>governorSettingsAreTheDeployedValues</code>  passes. In every state of the Governor's storage, <code>votingDelay</code> is 0, <code>votingPeriod</code> is 22 hours, <code>proposalThreshold</code> is 1,000 TERM, the quorum at any past timepoint is 1/100 of TERM's total supply at that timepoint, the executor is the Governor itself (no timelock), and no proposal needs queuing. None of these reads storage, so nothing the Governor stores can change them. The voting token is an immutable, and <code>name</code> is the only setting in storage, written only by the constructor. All four run on the deployed Governor (<code>0x2B715634134220ffeEE9458b4e34E41A41418607</code>) from its verified source, through <code>confs/Governor-settings.conf</code>.